

Part 8: Advanced Topics & Best Practices

Teaching Agentic Coding Tools to Students

Graduate Seminar

University of South Carolina

Spring 2026

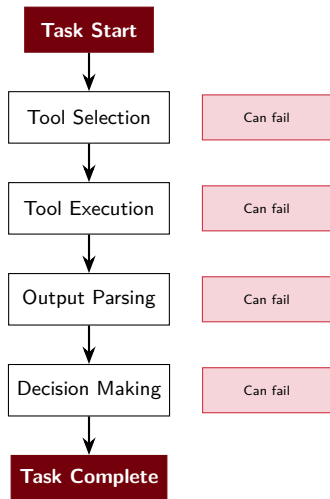
Table of Contents

- 1 Error Handling & Recovery
- 2 Security & Safety
- 3 Cost Management & Optimization
- 4 Testing Agentic Systems
- 5 Integration Patterns
- 6 Real-World Case Studies & Conclusion

When Agents Fail: Common Failure Points

Agents can fail at multiple stages:

- **Tool execution failure:** Tool crashes or returns unexpected format
- **Decision-making error:** Agent chooses wrong tool or parameters
- **Token exhaustion:** Context window fills before completion
- **Network/API issues:** External service unavailable
- **Timeout:** Task takes too long
- **Hallucination:** Agent invents non-existent tools



Managing Timeouts in Long-Running Tasks

The Problem:

User starts task -> Agent processes
-> 5 minutes pass -> TIMEOUT
Work is lost. Must restart.

Solution 1: Checkpointing

```
def run_with_checkpoints(task):  
    for chunk in split_task(task):  
        result = run_with_timeout(  
            chunk, timeout=60)  
        save_checkpoint(result)  
        if failed(result):  
            load_last_checkpoint()  
            break  
    return combine_results()
```

Solution 2: Simplification

Large task -> Timeout
-> Reduce scope -> Retry

Key Insight

Break work into natural boundaries:

- By file
- By function
- By feature
- By batch size

Recovering from Interruptions and Crashes

Recovery Pattern - With Git:

```
# During task: commit frequently
git add specific_files.py
git commit -m "checkpoint: X done"

# After interruption:
git log --oneline -10
git status # See what's incomplete

# Resume with context
```

Resumption Prompt:

```
We stopped work on this project.
Here's where we left off:
- Completed: [list]
- Remaining: [list]
Continue from where we stopped.
```

Recovery Pattern - Checkpoint File:

```
{
  "task": "Refactor auth",
  "completed": [
    "Analyzed existing code",
    "Identified functions",
    "Refactored User class"
  ],
  "remaining": [
    "Refactor Token class",
    "Refactor Session class",
    "Write tests"
  ],
  "last_output": "..."
}
```

Best Practice

Version control is your best recovery mechanism.

Understanding Why Agents Make Certain Choices

Debugging Agent Decision-Making

Strategy 1: Verbose Logging

```
# Request verbose output
"Run with verbose
logging so I can see
your reasoning"

# Look for:
# - Tool selection
# - Parameter choices
# - Rejected approaches
```

Strategy 2: Thinking Prompts

```
Before you write code,
think through:
1. What is the problem?
2. What are 3 approaches?
3. Which is best? Why?
4. What are pitfalls?
Then implement.
```

Common Decision Errors:

- Inefficient tool for task
- Incorrect parameters
- Wrong file/location
- Misunderstood requirement
- Over-complicated solution

Debugging Mindset

"What would I need to know to make this decision correctly?"

Managing Context Window Exhaustion

Token Usage Timeline:

```
Task Start: 2K tokens (prompt + setup)
5 Tool calls: +15K tokens (accumulate)
10 More calls: +30K tokens (growing)
Large file read: +20K tokens
Continue reading: +50K tokens
Agent tries: "Context window exceeded"
```

Model Limits:

- Claude 3.5 Haiku: 200K tokens
- Claude 3.5 Sonnet: 200K tokens
- Claude Opus 4.5: 200K tokens

Recovery Strategy:

```
# Summarize context
"Summary of completed work:
- Created auth system
- Added 3 middleware
- Fixed 2 bugs

Now we need to: [task]"
```

Estimation

- Code file: 5-50 tokens/line
- 1000-line file: 5-50K tokens
- Git diff: variable, large

Recognizing Repeated Failure Modes

Pattern 1: The Infinite Loop

- Agent adds logging to debug
- Agent removes feature to fix
- Agent reverts changes
- Back to original bug...

Fix: Set max iterations. After 3 fails, human intervention.

Pattern 2: Wrong Tool Syndrome

- Debug Python with Bash
- Modify JS with Python tool
- Build with wrong compiler

Fix: Explicitly state which tools are appropriate.

Pattern 3: Assumption Failure

- Agent assumes file exists
- Doesn't check first
- "Error: file not found"

Fix: Always verify preconditions.

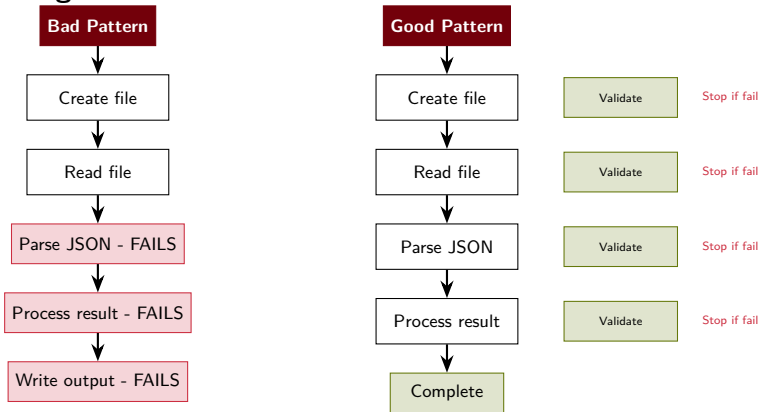
Pattern 4: Scope Creep

- "I'll fix the bug"
- "While here, I'll refactor"
- "Actually, let me add features"
- Task never completes

Fix: "Fix ONLY the bug. Don't refactor."

Preventing One Failure from Causing Many

Handling Cascading Failures



Principle

A failure in step 2 should not affect step 4. Validate at each checkpoint.

Tools and Techniques for Getting Back on Track

Recovery Strategies

Strategy 1: Checkpoint & Restore

```
# Save state
git add . && git commit -m \
    "checkpoint before change"

# If something goes wrong
git reset --hard HEAD~1
git log --oneline -5

# Resume with new approach
```

Strategy 2: Rollback & Retry

- Did it work? → Move forward
- Same error? → Different approach
- New error? → Fix that first
- Unclear? → Get more info

Strategy 3: Reduce Scope

```
Original: "Build complete auth"
Failed: "Too many parts"
New: "Build ONLY login endpoint"

Once that works:
"Add user registration"
"Add password reset"
```

Recovery Checklist:

- ☐ Understand what failed and why
- ☐ Assess if similar to previous
- ☐ Determine if scope should reduce
- ☐ Provide additional context
- ☐ Set iteration limit

What NOT to Do When Agents Fail

Anti-Patterns to Avoid

Anti-Pattern 1: Ignoring the Error

Bad: "Just continue anyway"

Better: "The file wasn't found. Let me tell you where it is."

Anti-Pattern 2: Over-Specifying Recovery

Bad: "Do X. If fails do Y. If fails do Z..."

Better: "Try X. If fails, stop and tell me."

Anti-Pattern 3: Silent Failures

Bad: Agent silently continues

Good: Agent reports what succeeded/failed

Anti-Pattern 4: Changing Strategy Mid-Stream

Bad: "Refactor" → "Rewrite" → "Just fix bug"

Better: Plan the strategy upfront.

Anti-Pattern 5: Assuming Agent Knowledge

Bad: "Fix following RESTful principles"

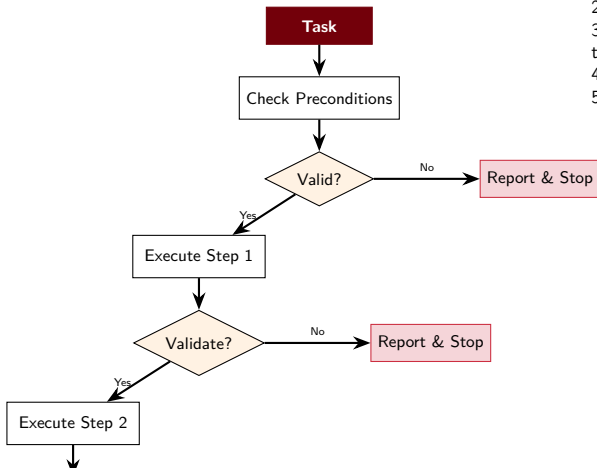
Better: "Fix following RESTful principles. Here's an example: [code]"

Designing for Failure from the Start

Building Fault-Tolerant Workflows

Principles:

1. Assume failure is possible
2. Make failures visible
3. Preserve state (version control)
4. Design incremental tasks
5. Validate at each step



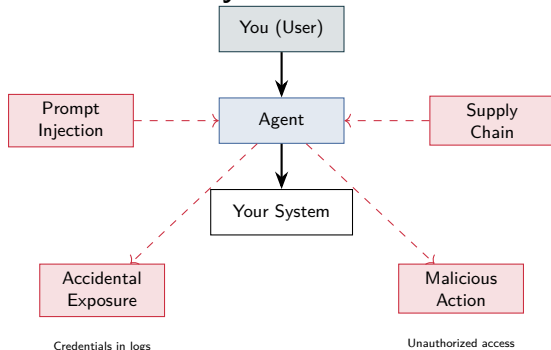
Agents Have Full System Access: What Could Go Wrong?

Understanding Agent Security Risks

Critical Reality: When you give an agent access:

- Read all your files (including keys, secrets)
- Modify any file
- Execute any command
- Delete files
- Network requests (download/upload)
- Install software
- Create backdoors
- Exfiltrate data

Security Threat Model



Trust Model

Strategies to Prevent Harmful Agent Actions

Preventing Malicious Code Execution

Strategy 1: Explicit Authorization

```
"Before executing commands  
that modify files, ask me  
for approval and show the  
exact command you plan to run."
```

Strategy 2: Restricted Scope

```
# Better:  
"You can run: git, python, npm  
You CANNOT: sudo, rm, install  
packages without asking"
```

Strategy 3: Command Whitelisting

- ✓ ls, cd, pwd (navigation)
- ✓ cat, grep (reading)
- ✓ python, node (interpreters)
- ✓ git (version control)
- ✓ pytest, npm test (testing)

- ✗ sudo (privilege escalation)
- ✗ rm -rf (destructive)
- ✗ curl (might exfiltrate)
- ✗ pip install (malicious pkg)

Principle

Make attacks expensive and obvious.

Protecting Secrets While Using Agents

Handling Sensitive Data

Strategy 1: Environment Separation

```
# Before invoking agent:
unset AWS_ACCESS_KEY_ID
unset AWS_SECRET_ACCESS_KEY
unset DATABASE_PASSWORD

# Invoke with limited env:
env -i HOME=$HOME USER=$USER \
    SHELL=$SHELL agent-command
```

Strategy 2: Use .claudeignore

```
# .claudeignore
.env
.env.local
.aws/
.ssh/
private_keys/
secrets/
```

Strategy 3: Redact Sensitive Info

```
# BAD:
DATABASE_URL=postgresql://
    admin:password123@
    db.prod.company.com/prod

# GOOD:
DATABASE_URL=postgresql://
    admin:REDACTED@
    example.com/example_db
```

Strategy 4: Temporary Credentials

```
# Generate temporary creds:
aws sts get-session-token \
    --duration-seconds 900
# Expires after 15 minutes
```

Writing Prompts That Can't Be Exploited

Safe Prompting Practices

Principle 1: Explicit is Better

```
# Weak:
"Add validation to this endpoint"

# Strong:
"Add validation to POST /users:
- Validate email format
- Password >= 8 characters
- Name is not empty
Do NOT change endpoint URL.
Do NOT change response format.
Do NOT add new endpoints."
```

Dangerous Prompts to Avoid:

- ✗ "Do whatever you think is best"
- ✗ "Make this awesome"
- ✗ "Optimize everything"
- ✗ "Fix all the issues"
- ✗ "Make it production-ready"
- ✗ "I trust you to handle this"

Better:

"Reduce query time by:

1. Adding indexes to user_id
2. Caching frequent queries

Test with: `pytest tests/performance/`"

Pre-Flight Security Review

Security Checklist for Agentic Workflows

Code & Data Security:

- ☐ Review prompt for unusual instructions
- ☐ Confirm no hidden instructions
- ☐ Verify no sensitive data exposed
- ☐ Check .gitignore/.claudeignore
- ☐ Ensure credentials not in env

System Access Control:

- ☐ Agent has only needed access
- ☐ Cannot execute arbitrary commands
- ☐ Cannot install packages
- ☐ Cannot access ~/.ssh or ~/.aws
- ☐ Network restricted

Monitoring & Recovery:

- ☐ Can see agent commands
- ☐ Can see file modifications
- ☐ Can interrupt immediately
- ☐ Have rollback plan
- ☐ Recent git commit exists

Example: Pre-Flight Check

Responding to Security Incidents with Agents

Incident Response

Scenario: Agent Exposed Credentials

Immediate (5 min):

1. Kill the agent process
2. Note what was exposed
3. Screenshot the response

Short-term (30 min):

1. Rotate all exposed credentials
2. Check CloudTrail for unauthorized access
3. Check git history for committed keys

Long-term (24 hr):

1. Audit similar configurations

Incident Report Template:

Date: 2026-01-23

Time: 14:30 UTC

What happened:

[Description]

How it was discovered:

[Alert **source**]

Immediate actions taken:

- [Action 1]

- [Action 2]

Credentials affected:

- [List]

Root cause:

[Why it happened]

Your Agent Security Rules

Building a Personal Security Policy

MY AGENT USAGE POLICY

Updated: [Date]

1. ALLOWED AGENTS

- Claude Code (terminal)
- Aider (code editor)

2. ALLOWED TOOLS

- [x] File read/write
- [x] Terminal: `git`, `python`, `npm`
- [] Network requests: limited
- [] Package install: NO

3. FORBIDDEN ACTIONS

- Execute `in` `/etc` or `/root`
- Access `~/.ssh` or `~/.aws`
- Network outside approved list
- Install packages
- Modify outside `/project/`

4. CREDENTIALS HANDLING

- Keep `.env` `in` `.claudeignore`
- Rotate before agent work
- Use temporary credentials
- Never include `in` prompts

5. MONITORING REQUIRED

- Before: Review agent access
- During: Watch `for` unusual
- After: Audit `git` diff

6. INCIDENT RESPONSE

If something goes wrong:
[Steps to follow]
Report to: [Contact]

7. REVIEW SCHEDULE

Review: Monthly

Signed: [Your name]

Keeping Agent Usage Secure Over Time

Long-Term Security Maintenance

Monthly:

- Review credentials in system
- Audit `~/.aws`, `~/.ssh`, `.env` files
- Verify `.gitignore` and `.claudeignore`
- Check git log for committed secrets

Quarterly:

- Review incidents from past 3 months
- Update personal security policy
- Learn from problems
- Share lessons with team

When Onboarding New Tools:

1. Evaluate: What access does it need?
2. Restrict: Give minimum necessary
3. Monitor: Set up logging/alerting
4. Document: Add to policy

Security Culture:

- ✗ "Security is IT's job"
- ✓ "Security is everyone's responsibility"

Indicators of Good Culture

People ask "is this secure?" before proceeding. Incidents are discussed openly.

Understanding Agent Costs

Cost Structure (per 1M tokens):

	Input	Output
Haiku	\$0.80	\$4.00
Sonnet	\$3.00	\$15.00
Opus	\$15.00	\$60.00

Real-world Example:

- Prompt: 5,000 tokens
- Tool calls: 20,000 tokens
- Response: 8,000 tokens
- Total: 33,000 tokens
- Cost (Haiku): ~\$0.12

Where Costs Hide:

1. Context Accumulation

First exchange: 5K → 10th exchange: 50K+

2. Tool Outputs

Read large file: +10K tokens

Run complex command: +5K per output

3. Debugging Iterations

3 attempts = 3x base cost

4. Retries & Restarts

Bad prompt: 20K wasted

New session: Another 20K

Measuring What You're Actually Spending

Token Usage Tracking and Monitoring Claude API (Programmatic):

```
from anthropic import Anthropic

client = Anthropic()
response = client.messages.create(
    model="claude-3-5-sonnet",
    max_tokens=1024,
    messages=[{"role": "user",
                "content": "Hello"}]
)

print(f"Input: {response.usage
        .input_tokens}")
print(f"Output: {response.usage
        .output_tokens}")
```

Monthly Tracking:

Week	Cost
Week 1	\$15.42
Week 2	\$12.80
Week 3	\$18.92 (spike)
Week 4	\$8.44
Total	\$55.58

Alert Thresholds:

- Daily: >\$5 (unusual)
- Weekly: >\$30 (intensive)
- Monthly: >\$150 (project-focused)

Writing Cheaper Prompts

Optimizing Prompts for Efficiency

BAD PROMPT (Many iterations):

```
User: "Help me fix this code"
Agent: "What code? What's wrong?"
User: "Python authentication"
Agent: "What specifically?"
User: "Password hashing not working"
Agent: "Here's how to debug..."
Total: ~50K tokens for confusion
```

GOOD PROMPT (One iteration):

```
"Fix password hashing in Python
auth module.
Currently: Using MD5 (insecure)
Should be: Using bcrypt
Files: auth/hash.py, auth/verify.py
Tests: tests/test_auth.py"
Total: ~15K tokens, done
```

Efficiency Principles:

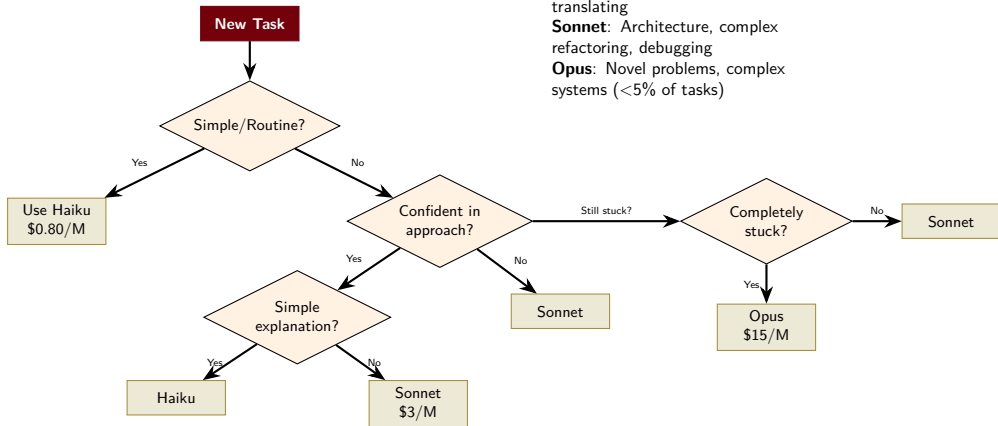
- 1. Be Specific Upfront**
All info in first message
- 2. Provide Examples**
Show don't tell
- 3. Specify Output Format**
"Format as bullet list only"
- 4. Remove Unnecessary Context**
Only what's needed

Result

Same tokens, 4x faster, clearer output.

When to Use Cheap vs Expensive Models

Choosing Models for Your Task



Controlling and Planning Agent Costs

Budget Management Strategies

Strategy 1: Tiered Budgets

```
Personal learning: $20/month  
Production work: $100/month  
Research: $50/month  
Total: $170/month  
Alert if > $200/month
```

Strategy 2: Pre-Commitment

```
"This task might require:  
- 3-4 exchanges  
- ~30K tokens each  
- Model: Sonnet  
- Est. cost: $0.50-0.80
```

```
Is this within budget? Y/N"
```

Strategy 3: Rate Limiting

- "Max 5 agent interactions per day"
- Prevents overuse through habit
- Forces better planning

Strategy 4: Batch Processing

```
# Bad: 3 separate interactions  
Task 1: "Fix lint" - 5K  
Task 2: "Fix lint" - 5K  
Task 3: "Add comment" - 5K  
Total: 15K, 3 interactions  
  
# Good: 1 combined interaction  
"Fix these 3 lints and add  
this comment" - 12K  
Total: 12K, 1 interaction  
Savings: 20%
```

Comparing Costs: Claude Code vs Aider vs Others

Cost Comparison Across Tools

Tool	Model	Cost Model
Claude Code	Haiku/Sonnet	API pricing
Aider	Haiku/Sonnet	API pricing
GitHub Copilot	GPT-4	\$20/month
ChatGPT Plus	GPT-4	\$20/month
Amazon Q	Claude	AWS pricing

Hybrid Strategy:

Claude Code

Specific tasks, occasional use, track costs

ChatGPT Plus

Learning, research, quick questions

Copilot

Code completion, inline suggestions

Total: ~\$30-50/month

Coverage: All needs

Small Bug Fix Cost (~20K tokens):

- Claude Code (Haiku): \$0.032
- ChatGPT Plus: \$0.67 per query avg

Pro Tips for Minimizing Agent Costs

Advanced Cost Optimization

Tip 1: Use Git, Not Conversation

```
# BAD: Agent outputs all changes
Agent: [Large code output]
Cost: High (all in tokens)

# GOOD: Agent commits directly
Agent: "Done, changes committed"
Cost: Low (minimal output)
```

Tip 2: Reuse Conversation Context

```
# BAD: New conversation each task
Each starts over, no memory

# GOOD: One conversation
"In previous task we did X.
Now do Y."
Savings: 30-50%
```

Tip 3: Checkpoint and Prune

```
# After major milestone:
git commit -m "Phase 1 complete"

# Start fresh
"We completed auth. Now
build the API layer."
Fresh start: 10K vs 100K
```

Optimized Approach Example:

- Session 1: Build API (25K)
- Session 2: Add auth (15K)
- Session 3: Fix bugs (15K)
- Session 4: Add tests (15K)
- **Total: 70K = \$0.25**

Should You Use an Agent for This Task?

ROI and Decision-Making Framework

Decision Framework:

1. **Well-understood task?**

Yes → Agent will do well, cheap

No → Might be expensive

2. **How much planning needed?**

Know what to do → Use agent

Need to figure out → Plan first

3. **What's your time worth?**

High (\$50+/hr) → Use agent even if \$0.50

Low (\$15/hr) → Only if saves >\$1

4. **Learning opportunity?**

Yes → Maybe do manually

No → Use agent

ROI Calculation:

$$\text{ROI} = (\text{Time saved} \times \text{Rate}) - \text{Cost}$$

Example:

- Task: Refactor auth
- Time without agent: 3 hrs
- Time with agent: 45 min
- Time saved: 2.25 hrs
- Rate: \$50/hr
- Agent cost: \$0.75

$$\begin{aligned}\text{ROI} &= (2.25 \times \$50) - \$0.75 \\ &= \$112.50 - \$0.75 = \mathbf{+\$111.75}\end{aligned}$$

Why Testing Agents is Different

Challenges of Testing Agent Behavior

The Challenge:

Traditional	Agent
Known input	Complex prompt
Deterministic	Variable output
Clear pass/fail	Ambiguous result

Non-Determinism:

- Same prompt, Run 1: Approach A
- Same prompt, Run 2: Approach B
- Both might be valid!

Vagueness in Requirements:

- Traditional: "add(2,3) returns 5"
- Agent: "Refactor to be better"
 - Faster?
 - More readable?
 - More maintainable?
 - All of above?

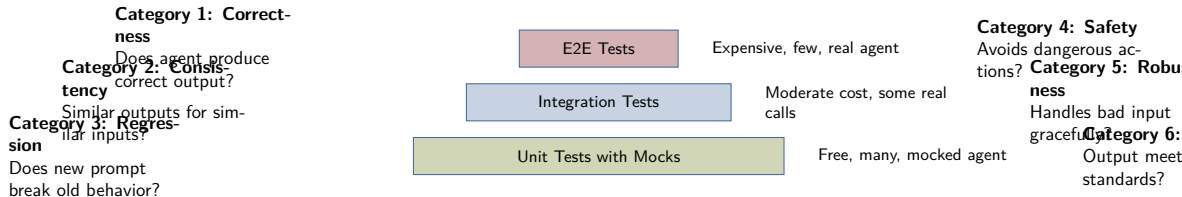
Cost of Testing:

- Each test run costs money
- 100 runs: \$10-100
- Can't run 1000 times

Key Insight

Different Tests for Different Aspects

Test Categories for Agent Systems



Writing Tests Before Asking Agent

Test-Driven Development with Agents

Principle: Define Success Upfront

```
# test_auth.py (write BEFORE)

def test_password_hashing():
    from auth import hash_password
    password = "test_password_123"
    hashed = hash_password(password)

    # Verify it's hashed
    assert hashed != password

    # Verify bcrypt format
    assert hashed.startswith("$2")

    # Verify verification works
    assert verify_password(
        password, hashed) == True
    assert verify_password(
        "wrong", hashed) == False
```

Then show agent the test:

```
"Here's my test suite:
[Tests above]

Please refactor the auth module
to pass these tests.
Run pytest to verify."
```

Agent Response Cycle:

1. Agent reads tests
2. Agent attempts refactoring
3. Agent runs pytest
4. Tests fail: Agent sees why
5. Agent fixes issues
6. Tests pass: Complete!

Preventing Agent Changes from Breaking Things

Regression Testing for Agent Workflows

Baseline Testing Process:

1. Establish baseline

- Run current workflow
- Document results
- Save outputs, metrics

2. Make changes

- Update prompt/model
- Update parameters

3. Run again

- Compare to baseline
- Document differences

4. Regression check

- Did new version break anything?
- Any unexpected changes?

Version Comparison:

Metric	V1	V2	Change
Success rate	95%	92%	-3%
Avg tokens	25K	24K	-1K
Avg time	2m	2.5m	+30s
Quality	8/10	8.2/10	+0.2
Test pass	98%	99%	+1%

Regression detected? No significant regressions

Decision: Deploy version 2

Continuous Regression

Run on each commit. Alert if any metric drops $>5\%$.

How to Know if Agent Output is Good

Validating Agent Output Quality

Quality Dimensions:

Dimension	Points
Correctness	0-30
Readability	0-20
Documentation	0-20
Security	0-20
Performance	0-10
Total	0-100

Acceptance Criteria:

- 0-39: Not acceptable, redo
- 40-69: Acceptable with fixes
- 70-84: Good quality

Scoring Example:

```
def total_score(code):  
    return sum([  
        correctness(), # Tests pass?  
        readability(), # Clear names?  
        documentation(), # Docstrings?  
        security(), # No vulns?  
        performance() # Efficient?  
    ])
```

Manual Review Checklist:

- ☐ Does it actually work?
- ☐ Are variable names clear?
- ☐ Any security issues?
- ☐ Any performance issues?
- ☐ Tests added/updated?

Testing Agents Without Running Expensive Operations

Mocking and Simulation for Agent Testing

Mock the Agent:

```
from unittest.mock import Mock, patch

def test_workflow_with_mock():
    # Create mock agent
    mock_agent = Mock()
    mock_agent.analyze_code.return_value = {
        "issues": ["SQL injection"],
        "severity": "HIGH"
    }

    # Patch real agent with mock
    with patch('mycode.agent',
               mock_agent):
        result = run_workflow(code)

        assert result["issues"] == 1
        assert result["severity"] == "HIGH"
```

Cost Comparison:

Approach	Tests	Cost
Mock testing	1000	\$0
Real agent	10	\$0.50-1

Best Practice:

- Use mocks for 99% of tests
- Use real agent for 1% critical tests

Simulate Different Behaviors:

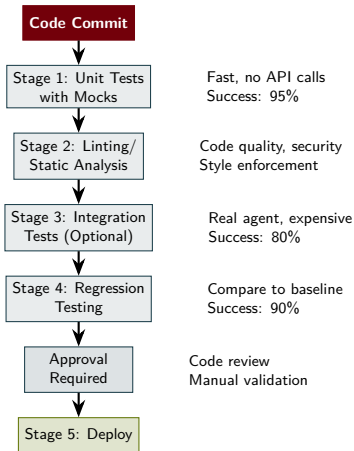
- Agent success
- Agent failure
- Agent timeout
- Agent hallucination

Automated Testing in CI/CD Pipelines

Continuous Integration for Agent Workflows

Best Practices:

- ✓ Use mocks for most tests
- ✓ Track costs
- ✓ Run on every commit
- ✓ Fail fast
- ✓ Set up cost alerts
- ✗ Run expensive tests on every commit
- ✗ Ignore test failures
- ✗ Let costs grow unchecked



Tracking Agent Behavior Over Time

Monitoring and Observability in Production

What to Monitor:

Availability

Agent online/offline, API response time, error rates

Quality

Task success rate, output quality scores, user satisfaction

Cost

Tokens per task, total costs trending, cost per successful task

Dashboard Views:

1. Real-time status

Current task, API health, errors (1hr)

2. Daily trends

Success rate, avg tokens, cost, quality

3. Weekly summary

Total tasks, common errors, trends

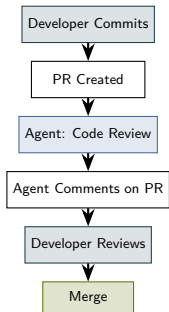
Alerts:

- Success rate $< 80\%$
- Cost per task increases $20\%+$
- Quality score $< 6/10$
- API error rate $> 5\%$

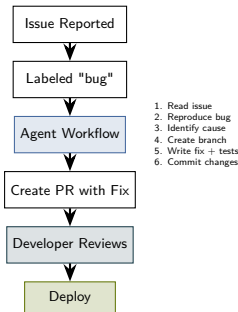
Agents as Part of Your Development Workflow

Combining Agents with CI/CD Pipelines

Pattern 1: Code Review Agent



Pattern 2: Bug Fix Agent



1. Read issue
2. Reproduce bug
3. Identify cause
4. Create branch
5. Write fix + tests
6. Commit changes

Agent in Different Stages:

Development:

Code generation, docs, testing

Testing:

Test generation, result analysis

Deployment:

Infrastructure, config, verification

Operations:

Monitoring, alerting, docs

Collaboration Patterns When Teams Use Agents

Using Agents in Team Workflows

Pattern 1: Shared Agent Prompts

- Team maintains: agent-prompts/
 - code-review.md
 - refactoring.md
 - documentation.md
- Same prompts → consistent results

Pattern 2: Agent Guardrails

- **Can:** read, analyze, suggest
- **Cannot:** commit, push, deploy without approval
- **Must:** provide clear explanations
- **Should:** flag security issues

Pattern 3: Agent Results Review

1. Agent generates code
2. Team reviews output
3. Humans decide: accept, modify, reject

Never: "Just run agent and deploy"

Team Onboarding:

1. How agents work (capabilities/limits)
2. Team agent usage policy
3. Standard prompts
4. How to review agent output
5. What to do if something goes wrong

Autonomy after 5-10 supervised tasks

Managing Code and Workflows Under Source Control

Version Control Best Practices with Agents

Good Git Practices:

```
# Commit frequently
git add auth.py
git commit -m "refactor: extract
password validation method"

# Before agent work
git checkout -b feature/auth

# Agent makes changes
# 1. Read code
# 2. Make changes
# 3. Run tests
# 4. Commit

# After verification
git push origin feature/auth
# Create Pull Request
```

Conventional Commits:

<type>: <description>

Types:

- feat: new feature
- fix: bug fix
- refactor: code reorg
- test: tests only
- docs: documentation
- perf: performance

Examples:

feat: add OAuth 2.0 support

fix: prevent SQL injection

refactor: simplify auth flow

Useful Commands:

- git diff HEAD (see changes)

Agents as Code Reviewers and Assistants

Code Review Integration

Scenario 1: Agent Reviews Human Code

1. Developer submits PR
2. Agent analyzes for:
 - Security issues
 - Performance problems
 - Style violations
 - Missing tests
 - Potential bugs
3. Agent comments on PR
4. Human reviewer makes final decision

Scenario 2: Human Reviews Agent Code

1. Agent generates code
2. Human reviews for:
 - Requirements match?
 - Quality acceptable?
 - Edge cases covered?
 - Design sound?
3. Decision: approve, modify, reject

Automated Tools (use alongside):

- Linting: eslint, pylint, clippy
- Security: bandit, sqlmap
- Type checking: mypy, TypeScript

Using Agents to Automate Terminal Workflows

Scripting and Terminal Automation

Scenario 1: Batch File Processing

```
# Agent-assisted approach:
"Write a script that:
- Reads all files in logs/
- Extracts ERROR or CRITICAL
- Aggregates into report.md
- Groups by error type
- Provides count and sample"

# Agent writes script
# You verify and run
# Result: 100 files in seconds
```

Scenario 2: Testing Automation

```
# Agent creates: run_tests.sh
echo "Running unit tests..."
pytest tests/unit/ -v --cov
echo "Running integration..."
pytest tests/integration/ -v
```

Scenario 3: Deployment

```
# Agent creates: deploy.sh
#!/bin/bash
set -e

echo "Pre-deployment checks..."
./scripts/pre-deploy.sh

echo "Running tests..."
npm run test

echo "Building..."
npm run build

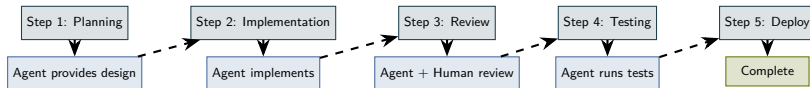
echo "Creating backup..."
git tag "backup-$(date +%Y%m%d)"

echo "Deploying..."
docker build -t app:latest .
docker push app:latest
```

Building a Complete Agentic Workflow

Real-World Integration Example

Feature: Add User Email Verification



What Humans Decided:

1. What problem to solve
2. Architecture approach (approved)
3. When to deploy
4. How to monitor
5. What to optimize

What Agent Handled:

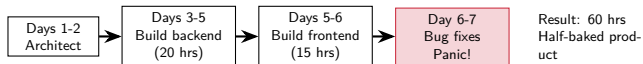
1. Generated code
2. Wrote tests
3. Checked for bugs
4. Provided documentation
5. Ran automated checks
6. Helped debug issues

Result: 2-4 hours (vs 1-2 days manual) | **Cost:** ~\$2 | **Ideal partnership:** humans decide, agents work

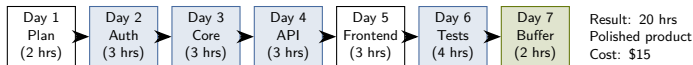
How Agents Accelerated a Startup MVP

Success Case Study - Rapid Prototyping

Without Agents (Traditional)



With Agents (Accelerated)



Key Success Factors: Clear requirements, experienced team lead, iterative approach, realistic expectations

Lessons: Agents excel at time-consuming, well-understood tasks. Human creativity and judgment still needed.

When Relying Too Much on Agents Goes Wrong

Failure Case Study - Over-Automation

The Problems:

Week 1: "Agent generated code, we deployed"

- ✗ Security vulnerability in production
- ✗ Customer data exposed
- Root cause: No code review before deploy

Week 2: "Use agent for all code"

- ✗ Token costs skyrocket
- ✗ \$500/month vs \$50 planned
- Root cause: No optimization

Week 3: "Agent refactored wrong part"

- ✗ Broke critical functionality

What Went Wrong:

1. No code review process
2. Over-trust in agent capability
3. No budget management
4. Vague requirements
5. No testing before deploy

Recovery (Month 2-3):

- Implement mandatory code review
- Define agent use cases explicitly
- Set budget limits
- Improve prompt clarity

Recognizing Limitations and Boundaries

When NOT to Use Agents

Don't Use Agents For:

1. **Strategic Decisions**
Agent can analyze, human must decide
2. **Exploratory Research**
If goal is understanding, do it yourself
3. **New Problem Domains**
Learning is necessary
4. **High-Risk Systems**
Medical, financial need expert review
5. **One-Time Complex Tasks**
Setup overhead > time saved
6. **When Speed Isn't Important**
No time pressure = no urgency for

When to Avoid:

No Time for Learning Curve

10 hrs to learn agents, task takes 5 hrs
→ Might not be worth it

No Verification Capability

Can't tell if output is correct
→ Risky to use agent

Security Concern

Sensitive code, credentials present
→ Risk > benefit

Ideal Use Cases for Maximum Value

When Agents Shine

1. Well-Understood Problems

- Common, documented patterns
- Clear requirements
- Examples: REST APIs, CRUD, refactoring

2. Time-Consuming, Repetitive Work

- Lots of boilerplate code
- Humans find tedious
- Examples: tests, migrations, configs

3. Exploratory and Prototyping

- Quick experimentation
- Low cost of being wrong

4. Review and Analysis

- Comprehensive, consistent reviews
- Examples: code review, docs, coverage

5. Context Gathering & Docs

- Summarizing large codebases
- Examples: README, API docs

6. Skill-Building & Learning

- Agent provides examples
- You learn by reviewing

Success Indicators

Well-understood, clear requirements, can verify, time saved, cost reasonable, risk

Where Agentic Tools Are Headed

Future Directions and Evolution

Near-term (6-12 months):

- Better specialization
- Improved reliability
- Less hallucination
- Better integration
- Cost reduction

Medium-term (1-2 years):

- Autonomous agents
- Better reasoning
- Multi-step workflows
- Team collaboration
- Multiple agents working together

Long-term (2+ years):

- Agents as colleagues
- Continuous improvement
- Natural language interfaces
- Cross-domain reasoning

Challenges:

- Safety at scale
- Cost efficiency
- Environmental impact
- Regulatory compliance

How to Become Expert in Using Agents

Building Your Agent Mastery

Phase 1: Fundamentals (Weeks 1-4) ✓ Completed!

- Understand capabilities/limits
- Learn clear prompts
- Practice terminal usage
- Understand cost model

Result: Use agents for simple tasks

Phase 2: Intermediate (Months 2-3)

- Optimize prompts
- Build complete workflows
- Integrate with CI/CD
- Handle failures gracefully
- Mentor others

Result: Use agents for complex tasks

Phase 3: Expert (Months 4+)

- Design agent workflows
- Optimize organization-wide
- Establish best practices
- Innovate new patterns
- Contribute to open source

Result: Thought leader

Your Next Steps

Week 1: Complete activities | Weeks 2-4: Use on real projects | Month 2: Tackle complex

Distilled Wisdom from Part 8

Key Takeaways and Principles

Principle 1: Tools, Not Magic

Right tool for right job. Be selective.

Principle 2: Failures Are Learning

Each failure teaches something. Document and share.

Principle 3: Clarity Beats Sophistication

Clear, simple prompts work best.

Principle 4: Verification Always Required

Agents make mistakes. Review and test.

Principle 5: Cost Awareness Matters

Track spending, optimize queries, budget.

Principle 6: Security is Your Responsibility

Set boundaries, restrict access, rotate credentials.

Principle 7: Human-Agent Partnership

Humans decide, agents work, humans verify.

Principle 8: Continuous Improvement

Iterate, learn, update, share.

These Principles Transcend Specific Tools

Whether using Claude Code, Aider, or tools not yet invented, these principles will serve you.

Mindset for Working with Agents

Final Thoughts: Agent Development Ethos

Collaborative Mindset:

- Think of agent as junior colleague
- Not: "Make it work"
- Think: "Here's what I need, can you help?"

Humble Approach:

- You might misunderstand
- Agent might misunderstand
- Misunderstandings are normal
- Iteration is expected

Experimentation Culture:

- Try different prompts
- Experiment with approaches
- Document findings
- Share learnings

Ethical Responsibility:

- Agents are powerful
- Power brings responsibility
- Use to create value
- Never use to harm

Where to Learn More

Resources and Further Reading

Official Documentation:

- anthropic.com/research
- docs.anthropic.com
- Claude API docs
- Aider GitHub

Key Papers:

- "Attention is All You Need"
- "Chain-of-Thought Prompting"
- "ReAct" paper
- "Constitutional AI"

Learning Resources:

- DeepLearning.AI courses
- Fast.ai NLP courses
- Andrew Ng's ML courses

Communities:

- Hugging Face community
- Papers With Code
- Local AI/ML meetups
- University AI clubs

Tools to Explore:

- Langchain (frameworks)
- LlamaIndex (indexing)
- Hugging Face (models)
- Cursor (IDE)
- Continue.dev (plugin)

Staying Updated:

- Anthropic newsletters
- The Batch
- AI podcasts
- Twitter/X AI accounts

Completing Part 8 and Your Agent Journey

Course Conclusion and What's Next

What You've Learned:

- ✓ Error Handling & Recovery
- ✓ Security & Safety
- ✓ Cost Management
- ✓ Testing Agentic Systems
- ✓ Integration Patterns
- ✓ Real-World Case Studies

You Are Now Ready To:

- Use agents in real projects
- Handle failures gracefully
- Manage costs and budgets
- Test agent behavior

Your Next Steps:

Immediate: Complete activities, implement security, set up tracking

Short-term: Use on real project, document experience, share learnings

Medium-term: Build production system, create case study, mentor someone

Long-term: Master agent-assisted development, lead adoption, become expert

A Final Word

The future of development is agent-assisted.

You're prepared to be part of that future.

Good luck. Go build something amazing.

END OF COURSE

Part 8: Advanced Topics & Best Practices

Teaching Agentic Coding Tools to Students

50 slides | 100+ code examples | Complete framework for agent use